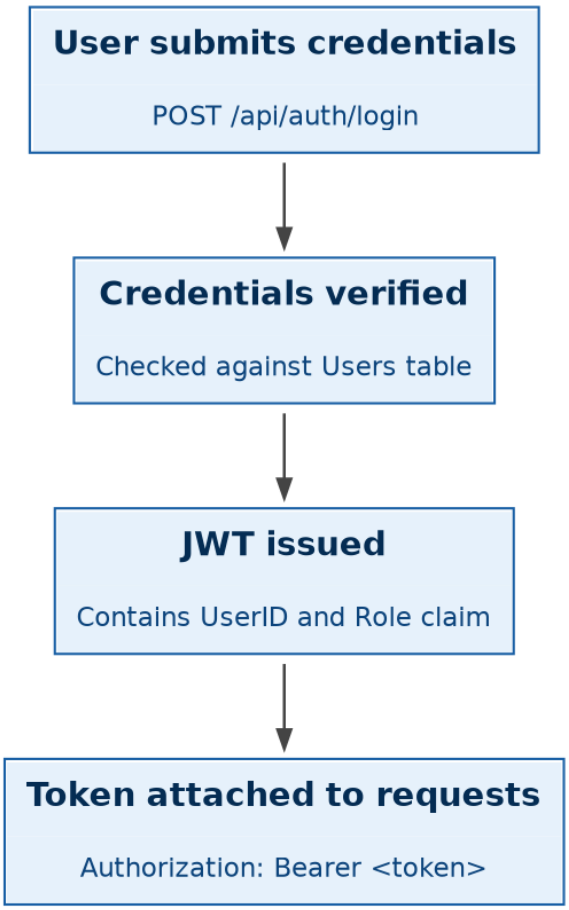
RaceDay System — Role-Based Access Design

This document defines the two roles in the RaceDay system (Organiser and Participant), how the API authenticates a user, and how each request is authorized once the user is known. It maps directly onto the Role Required column in the API Endpoint Plan (Section B) and the Role column on the Users table (Section C).

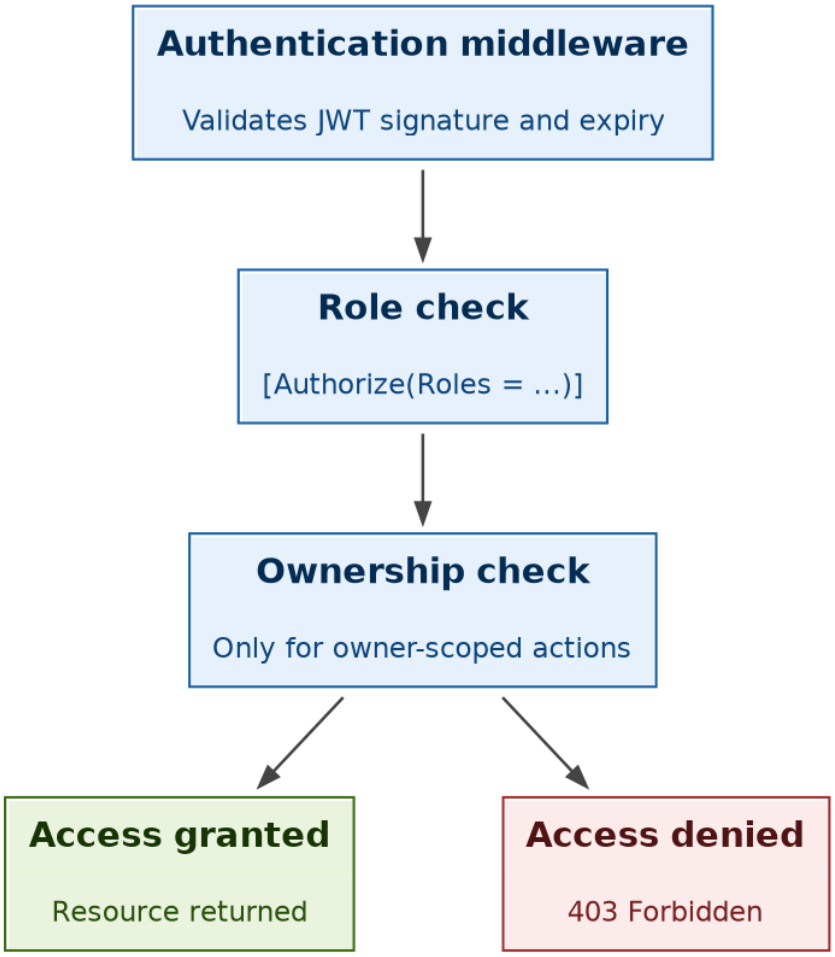
1. Authentication flow

A user logs in once and receives a signed token (JWT) that encodes their identity and role. That token is then attached to every subsequent request, so the server never needs to re-query the database just to know who is asking.



2. Authorization flow

On each request, the server runs three checks in sequence. The first two are always applied; the third (ownership) only applies to actions scoped to a specific resource, such as editing an Event or capturing a Result.



"Owner only" means two checks stack: the JWT role claim must match the required role, and the app must separately confirm that user's UserID matches the OrganiserID (or ParticipantID) tied to the resource being accessed. A role check alone would let any Organiser edit another Organiser's event, which is not the intended behaviour.

3. Permissions matrix

Restates the Role Required column from the Section B endpoint plan as a single lookup table.

Resource / action	Public	Participant	Organiser
Register / login	Yes	-	-
View own profile	-	Yes	Yes
View another user's profile	-	No	Yes
Browse events / categories	Yes	Yes	Yes
Create / update / delete an event	No	No	Yes (owner only)
Create / update / delete a category	No	No	Yes (owner only)
Enrol in a category	No	Yes	No
View own enrolments	No	Yes (own only)	-
View a category's enrolment roster	No	No	Yes (owner only)
Cancel an enrolment	No	Yes (own only)	No
Capture / correct a result	No	No	Yes (owner only)
View leaderboard / results	Yes	Yes	Yes

4. Implementation notes

• **Where the ownership check lives:** not inside [Authorize(Roles=...)] itself, which only checks role — implemented as a guard clause in the controller action, or a reusable IAuthorizationHandler if the same check is needed across multiple endpoints.

• **Chained ownership:** for Categories and Enrolments, "owner" is not a direct column — it is inherited through Category → Event → OrganiserID. The authorization query must join through that chain rather than checking a field on the Category itself.

• **Not enforced by the database:** nothing in the SQL schema (Section C) stops a Participant-role user from being inserted as an Event's OrganiserID, or vice versa. Role correctness is an application-layer responsibility, not a schema constraint.